

Versi 3.1
Agustus 2026



PEMROGRAMAN MOBILE

MODUL 1 - SINTAKS & UI DASAR

Penulis:

Eko Felix Nur Noftianto, M.Kom.

Wira Yudha Aji Pratama

Fatahillah Al-Fatih

LABORATORIUM INFORMATIKA
Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	4
1. TUJUAN.....	4
2. TARGET MODUL.....	4
3. PERSIAPAN.....	4
4. KATA KUNCI.....	4
MATERI.....	5
1. STRUKTUR DASAR REACT.....	5
CODELAB-1 (siapkan dari rumah).....	5
1.1. STRUKTUR IMPORT.....	6
1.2. RETURN KOMPONEN REACT.....	8
2. KOMPONEN.....	10
CODELAB-2 (5-10 menit, kerjakan di lab).....	12
3. STYLING DASAR.....	13
3.1. INTERNAL STYLING.....	13
3.2. EXTERNAL STYLING.....	16
3.3. INLINE STYLING.....	18
CODELAB-3 (5-10 menit, kerjakan di lab).....	19
4. PACKAGE, LIBRARY, & DEPENDENCY.....	20
CODELAB-4 (5-10 menit, kerjakan di lab).....	25
5. SINTAKS DASAR TYPESCRIPT.....	25
5.1. VARIABLE.....	25
5.2. CONDITIONS.....	29
5.3. FUNCTIONS.....	30
A. FUNCTION BAWAAN:.....	32
B. FUNCTION CUSTOM:.....	33
C. CALLBACK FUNCTION (BONUS).....	34
5.4. LOOPS.....	34
A. MAP().....	34
B. <FLATLIST> (BUKAN LOOP TAPI GUNANYA SAMA).....	37
C. PRIMITIVE LOOP.....	38
5.5. VARIABLE LANJUTAN.....	40
A. ARRAY OF OBJECT.....	40
B. TYPE & INTERFACE.....	41
CODELAB-5 (5-10 menit, kerjakan di lab).....	43
TUGAS PEKAN DEMO.....	45
1. DESKRIPSI.....	45





2. KRITERIA PENILAIAN..... 46

3. RENTANG PENILAIAN..... 46

KESIMPULAN AKHIR MODUL..... 47



PENDAHULUAN

1. TUJUAN

1. Memungkinkan mahasiswa memahami sintaks dasar dan konsep pemrograman yang digunakan dalam pengembangan React Native.
2. Memberikan pemahaman yang jelas kepada mahasiswa mengenai JSX/TSX serta komponen UI dasar dalam React Native.
3. Membekali mahasiswa dengan pengetahuan yang diperlukan untuk membuat dan menyusun antarmuka pengguna dasar menggunakan React Native.

2. TARGET MODUL

1. Mampu menggunakan sintaks dasar TypeScript dan tipe data dalam aplikasi React Native.
2. Mampu membuat dan menyusun UI menggunakan JSX/TSX.
3. Mampu menggunakan komponen dasar React Native seperti View, Text, Image, TextInput, Button, dan Pressable.
4. Mampu menerapkan styling dan layout dasar pada komponen React Native.

3. PERSIAPAN

1. Mahasiswa Muslim dapat membaca doa berikut sebelum memulai pembelajaran:

رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Laptop atau komputer pribadi (Windows/Mac).
3. Visual Studio Code.
4. Koneksi internet.
5. Telepon seluler (Android / iOS).
6. Motivasi dan komitmen yang kuat untuk belajar.

4. KATA KUNCI

Komponen, StyleSheet, Sintaks TypeScript, Dependency.



MATERI

Selamat datang di modul pertama Praktikum Pemrograman Mobile. Pada modul pertama ini, kamu akan mempelajari sintaks dasar React Native beserta struktur dan komponen-komponennya. Jangan khawatir, Modul 1 masih cukup mudah. Pastikan kamu menyelesaikan semua tugasnya!

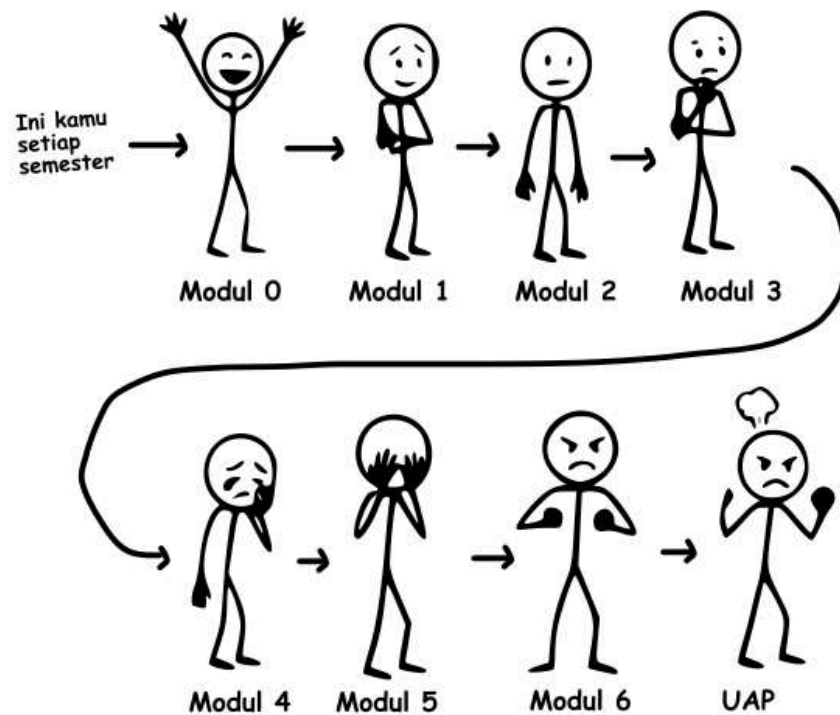


Image 0.1

1. STRUKTUR DASAR REACT

CODELAB-1 (siapkan dari rumah)

Yap, kamu benar. Kita langsung masuk ke Codelab 🐱! Di Modul 0, kita sudah menjelaskan cara membuat project baru dari awal, kan? Silakan ikuti langkah-langkah tersebut terlebih dahulu untuk membuat proyek. Setelah itu, pastikan struktur folder di dalam direktori **src/app** terlihat seperti berikut:

```
app/
└─ index.tsx
```

Itu saja untuk sekarang. Simpan project-mu dan jangan tutup dulu. Kita akan melanjutkan pengerjaannya pada CODELAB 2 dan bagian-bagian berikutnya. Jadi, baca modulnya sambil langsung mempraktikkannya. Untuk modul-modul selanjutnya, silakan siapkan project baru sebelum mulai praktikum agar saat di lab, environment sudah siap untuk codelab!



Oke, mari kita lanjutkan ke materi. Jika kamu membuka `/app/index.tsx` (yang merupakan entry point utama program), kamu akan melihat bahwa struktur dasar React Native terlihat seperti berikut:

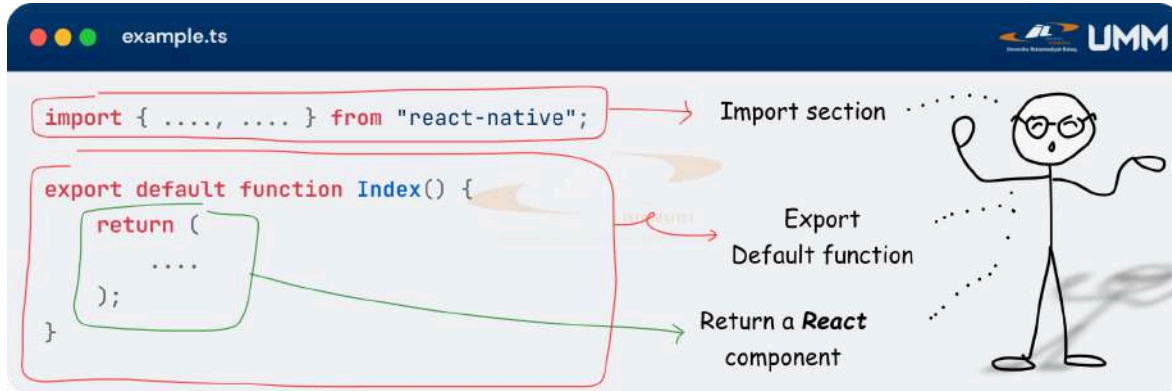


Image 1.1

Bagian import digunakan untuk mengimpor semua komponen yang kamu gunakan dalam kode.

Kita dapat menempatkan semua komponen di dalam bagian: **"return (...);"** yang berada di dalam **"export default function"**.

1.1. STRUKTUR IMPORT

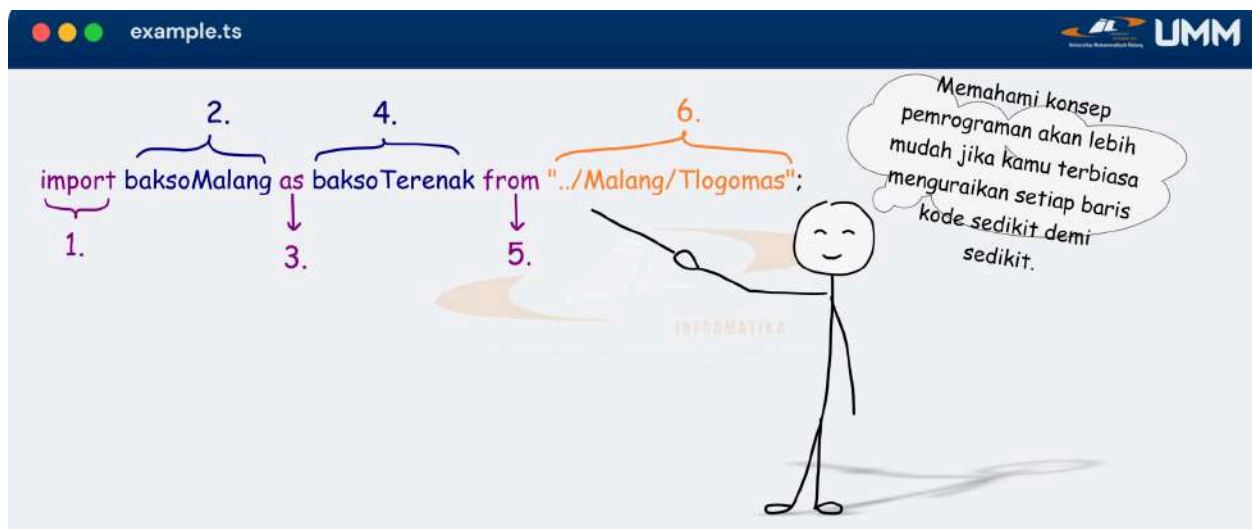


Image 1.1.1



Penjelasan:

1. Import:

- import adalah sebuah keyword yang digunakan untuk mengambil value, function, component, class, atau module dari file atau module lain agar dapat digunakan di file saat ini.

2. Apa yang di-import:

- Bagian setelah import menentukan apa yang ingin kita ambil dari paket tersebut. Biasanya, kita hanya dapat mengambil sesuatu dari file lain yang menggunakan keyword "export".

- Sebagai contoh, jika file atau package lain menggunakan:

```
export default function baksoMalang() {}
```

Maka kita dapat menggunakannya dengan:

```
import baksoMalang
```

Untuk mengimpor function tersebut ke file kita.

- Namun, jika file lain tidak menggunakan keyword default, misalnya:

```
export function baksoMalang() {}
```

maka kita menggunakan:

```
import { baksoMalang }
```

Ini disebut named export dan berbeda dari default export.

- Kita juga dapat menggunakan tanda asterisk (*) untuk mengimpor semua yang diekspor dari sebuah module.

```
import * as bakso
```

Ini berarti kita tidak perlu mengimpornya satu per satu, seperti:

```
import baksoMalang, { baksoUrut, baksoBeranak, baksoMercon }
```

3. As (opsional):

- Ini tidak wajib digunakan. as hanya digunakan untuk mengganti nama sesuatu yang kita impor menjadi nama apa pun yang kita inginkan atau yang menurut kita lebih nyaman digunakan.
- Namun, jika kita menggunakan `import *`, maka as menjadi wajib karena kita membutuhkan satu keyword atau nama untuk merepresentasikan semua komponen yang kita impor.

4. Bagian setelah as:

- Tuliskan bagian ini hanya jika kita menggunakan keyword "as". Kita bebas memberinya nama apa pun, selama nama tersebut tidak bertentangan dengan nama lain atau reserved keyword.
- Jika kita menggunakan namespace import dengan *, kita mengakses anggota-anggota yang diimpor melalui nama yang kita berikan.

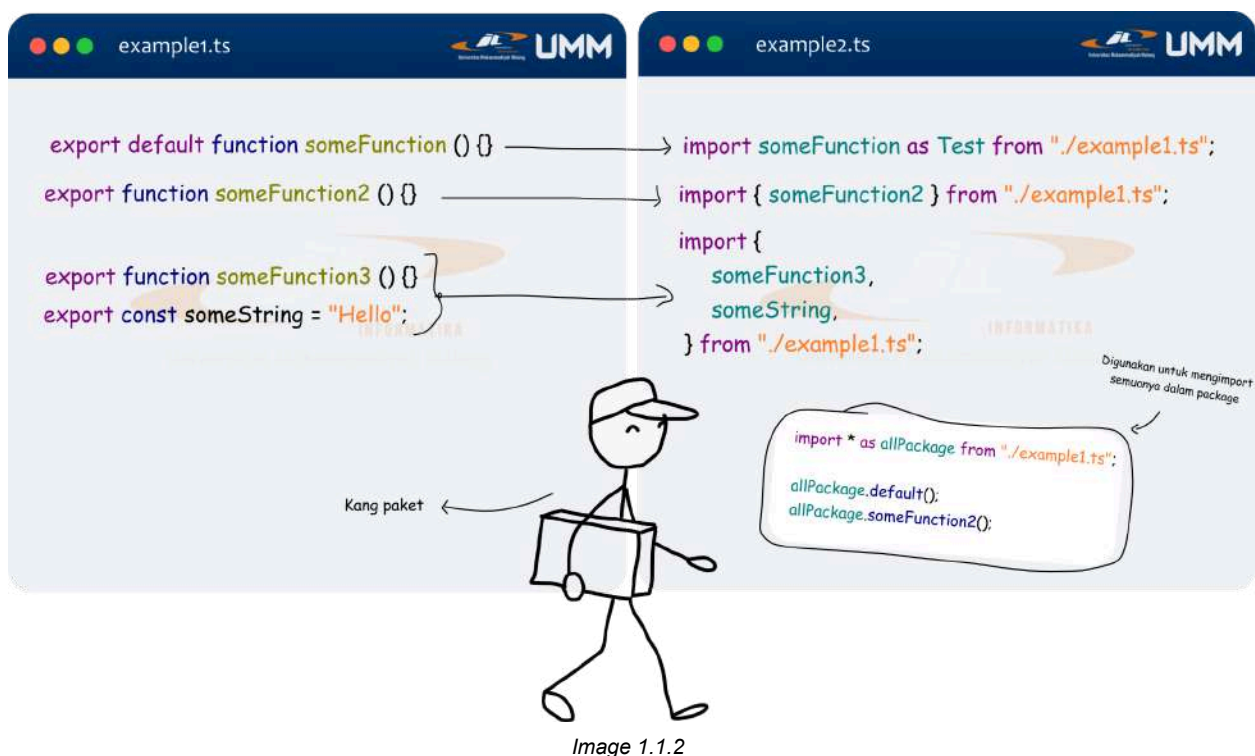


5. From:

- Keyword from menunjukkan dari mana module tersebut diimpor. Keyword ini diikuti oleh:

6. Location:

- Di sini, kita dapat langsung menuliskan nama module atau library yang ingin kita impor. Biasanya, module atau library tersebut berada di dalam folder `node_modules`.
- Namun, jika itu adalah local module yang kita buat sendiri, kita dapat menggunakan path seperti `./components/Button`.
- Kita juga dapat menggunakan `@` untuk merujuk ke direktori utama (root directory) proyek, seperti `@/app/profile.tsx`.



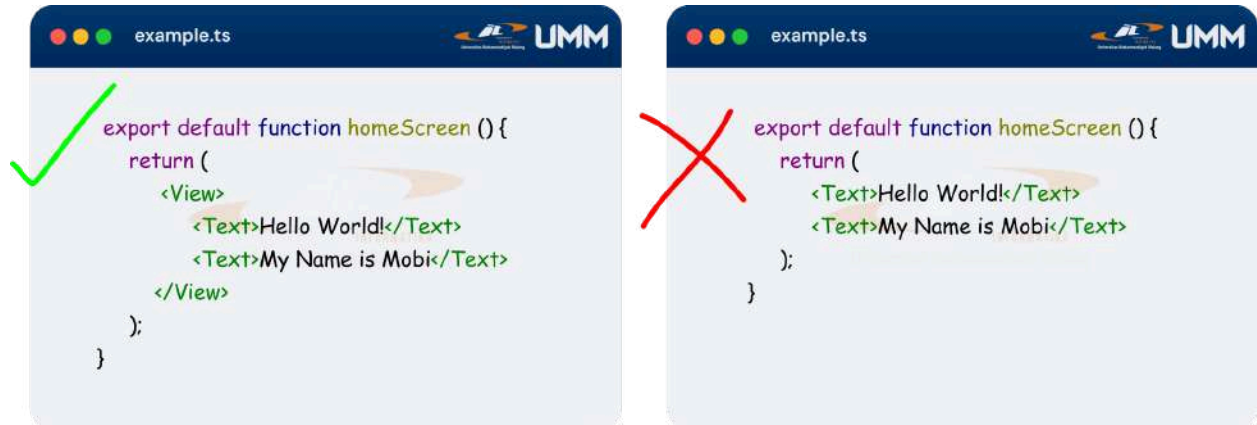
1.2. RETURN KOMPONEN REACT

React Native Expo menggunakan file-file yang berada di dalam `app/` sebagai routes. Di sini, `export default function` penting karena Expo Router memerlukan default export yang berisi sebuah React component untuk mengubah file tersebut menjadi sebuah screen/route. Nama function bebas ditentukan oleh kamu.

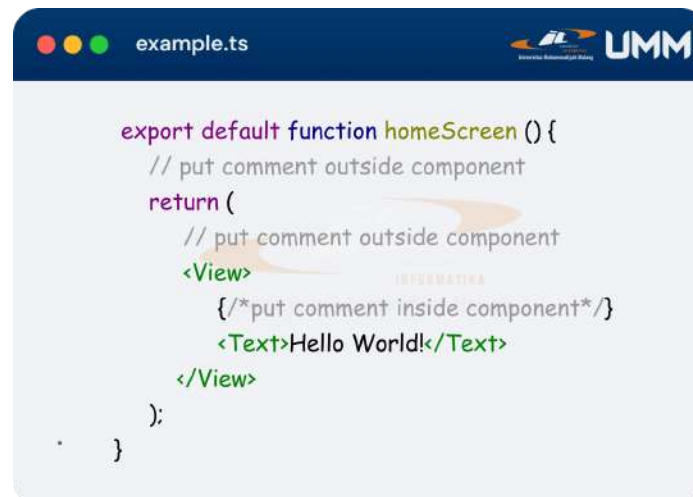
Kemudian, pada bagian return yang berisi komponen-komponen UI, hanya boleh ada satu root component; kita tidak boleh memiliki lebih dari satu.



Sebagai contoh, bentuknya adalah:



Kita juga dapat menambahkan komentar di dalam component dengan cara berikut:



Perhatikan bahwa sintaks { ... } di dalam component digunakan untuk masuk ke JavaScript mode. Kita juga dapat menggunakannya untuk mengakses variabel, melakukan operasi matematika, atau, dalam kasus ini, menambahkan komentar pada kode kita.



2. KOMPONEN

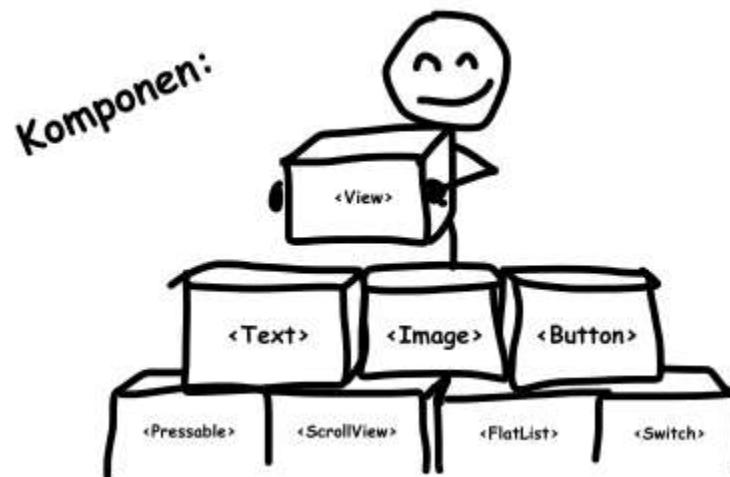


Image 2.1

Komponen dalam React Native adalah blok-blok pembangun yang digunakan untuk membangun antarmuka pengguna (UI) aplikasi. Setiap komponen memiliki fungsi tertentu dan dapat digabungkan dengan komponen lain untuk membangun layar yang lengkap.

Contoh:

```
example.ts
import { View, Text } from "react-native";

export default function homeScreen() {
  return (
    <View>
      <Text>Hello World</Text>
    </View>
  );
}
```

Output:

Hello World



Anda dapat melihat komponen-komponen utama yang umum digunakan di sini:

Tabel 2.1

Komponen	Function	Example Usage
View	Digunakan sebagai wadah atau pembungkus tata letak. Sama seperti <code><div></code>	<code><View> </View></code>
Text	Menampilkan teks di layar	<code><Text>Hello World!</Text></code>
Image	Menampilkan gambar	<code><Image source={{ uri: "https://picsum.photos/200" }} /></code>
Pressable	Membuat elemen yang dapat diklik/disentuh	<code><Pressable> <Text>Click Me</Text> </Pressable></code>
TextInput	Menerima input dari pengguna	<code><TextInput placeholder="Enter name" /></code>
ScrollView	Membuat layar yang dapat digulir	<code><ScrollView>...</ScrollView></code>
FlatList	Menampilkan daftar data secara efisien	<code><FlatList data={data} renderItem={({ item }) => <Text>{item}</Text>} /></code>

Dan masih banyak lagi. Untuk dokumentasi lengkapnya, Anda dapat membaca dokumentasi resmi React Native pada tautan di bawah ini atau menonton panduan lengkap untuk pemula di YouTube melalui tautan berikut.

Dokumentasi Resmi React Native: [Dokumentasi Resmi React Native](#).

Panduan Lengkap untuk Pemula di YouTube: [React Native Full Course for Beginners](#).



CODELAB-2 (5-10 menit, kerjakan di lab)

Lanjutkan dari Codelab 1 dan isi `/app/index.tsx` dengan kode di bawah ini (lengkapi bagian "____" yang masih kosong):

```
example.ts

import {
  View,
  Text,
  _____,
  Button,
} from "react-native";

export default function Index() {
  return (
    <View>
      <Text>Hello World</Text>
      <TextInput placeholder="Type here..." />
      <_____ title="Click Me" />
    </_____>
  );
}
```

Setelah selesai, jalankan proyek. Jika semuanya berjalan dengan benar, pastikan hasil tampilannya terlihat seperti ini:



Pada beberapa perangkat, warna latar belakang dan elemen visual lainnya mungkin terlihat berbeda karena tampilan biasanya disesuaikan dengan tema perangkat. Jika perangkat menggunakan tema gelap (dark theme), beberapa warna akan terlihat lebih gelap. Jika menggunakan tema terang (light theme), warna-warna tersebut akan terlihat lebih cerah.



3. STYLING DASAR

3.1. INTERNAL STYLING



Image 3.1.1

Dalam React Native, kita dapat menambahkan styling pada komponen menggunakan StyleSheet. Kita dapat menyesuaikan berbagai hal seperti:

- warna latar belakang
- ukuran teks
- jarak antar komponen
- posisi komponen
- sudut yang membulat

Biasanya, *styling* dibuat terlebih dahulu menggunakan `StyleSheet.create()`. Kemudian, *style* tersebut dapat diterapkan pada komponen tertentu.

Contohnya: `<View style={styles.container}>` Artinya, komponen View menggunakan style bernama "container" dari "styles". Tapi, style tersebut berasal dari mana? Nah, kita perlu membuat style-nya terlebih dahulu 😊.

Berikut caranya:

```
example.ts
import { View, Text, StyleSheet } from "react-native";

export default function Index() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>
        Hello World
      </Text>
    </View>
  );
}
```



```

    );
  }

  const styles = StyleSheet.create({
    container: {
      flex: 1,
      backgroundColor: "black",
      justifyContent: "center",
      alignItems: "center",
    },
    text: {
      fontSize: 30,
      color: "yellow",
      backgroundColor: "red",
      padding: 15,
      borderWidth: 3,
      borderColor: "lime",
      borderRadius: 10,
    },
  });

```

Output:



Berikut adalah beberapa atribut styling yang umum digunakan dalam React Native. Atribut-atribut ini merupakan atribut dasar yang sering kamu lihat dan gunakan saat membuat antarmuka pengguna.

Tabel 3.1.1

Attribute	Function	Example
backgroundColor	Mengubah warna latar belakang	"blue" or "#2563eb"
color	Mengubah warna teks	"white"
fontSize	Mengubah ukuran teks	20
fontWeight	Mengubah ketebalan teks	"bold"



padding	Menambahkan jarak di dalam komponen	20
margin	Menambahkan jarak di luar komponen	20
marginTop	Menambahkan jarak di atas komponen	10
paddingTop	Menambahkan jarak di bagian atas dalam komponen	20
borderRadius	Membuat sudut menjadi membulat	16
width	Mengatur lebar komponen	200 or "100%."
height	Mengatur tinggi komponen	100
alignItems	Menyelaraskan elemen secara horizontal	"center"
justifyContent	Menyelaraskan elemen secara vertikal	"center"
flex	Membuat komponen mengisi ruang yang tersedia	1
textAlign	Mengatur posisi teks	"center"
elevation	Menambahkan efek bayangan (Android)	5
shadowColor	Mengatur warna bayangan (iOS)	"#000"

Sebenarnya, ada banyak atribut styling lain yang dapat digunakan di React Native. Tidak mungkin membahas semuanya di sini 😊. Untuk saat ini, kita hanya akan berfokus pada atribut yang paling umum digunakan terlebih dahulu.

Kamu dapat melihat dokumentasi lengkapnya di sini: [Dokumentasi Style React Native](#).

Atau tonton tutorial videonya di sini: [Tutorial Styling React Native di YouTube](#)

Tidak perlu menghafal semuanya langsung. Kamu akan secara alami mempelajari lebih banyak atribut styling seiring dengan semakin banyak project yang kamu buat.

Jangan khawatir membuat kode terlihat sempurna dulu. Saat ini, tujuannya adalah memahami cara kerja styling.



3.2. EXTERNAL STYLING

Seperti namanya, External Styling berarti menuliskan style di file yang sepenuhnya terpisah, lalu mengimpornya ke file komponen utama (index.tsx).

Mengapa kita melakukan ini?

- **Clean Code:** File komponen tidak akan dipenuhi oleh ratusan baris aturan styling.
- **Reusable:** Kamu dapat menggunakan styling yang sama pada beberapa screen atau komponen tanpa perlu menulis ulang kode.

Berikut cara melakukannya:

Langkah 1: Buat file style (misalnya, styles.ts atau /constants/styles.ts)

```

import { StyleSheet } from "react-native";

export const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#f1f5f9",
    justifyContent: "center",
    alignItems: "center",
    padding: 24,
  },
  card: {
    backgroundColor: "ffffff",
    padding: 20,
    borderRadius: 16,
    width: "100%",
    alignItems: "center",
    elevation: 4,
    shadowColor: "#000",
  },
  title: {
    fontSize: 22,
    fontWeight: "bold",
    color: "#0f172a",
    marginBottom: 8,
  },
  subtitle: {
    fontSize: 14,
    color: "#64748b",
  },
});
  
```



Langkah 2: Import dan gunakan file tersebut di komponen (app/index.tsx)

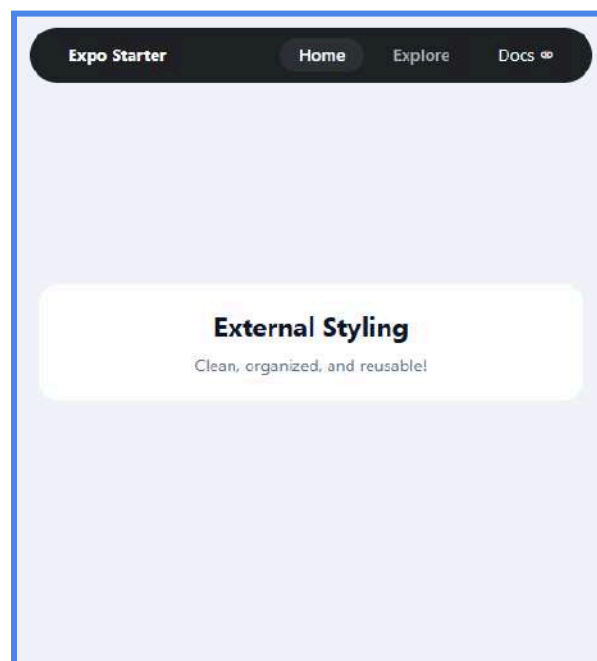
```

index.tsx

import React from "react";
import { View, Text } from "react-native";
import { styles } from "../styles"; // Import the external styles we just created

export default function HomeScreen() {
  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.title}>External Styling</Text>
        <Text style={styles.subtitle}>Clean, organized, and reusable!</Text>
      </View>
    </View>
  );
}
  
```

Output:



Sebuah card yang rapi dan berada di tengah layar dengan pemisahan yang jelas antara logika dan styling. Tidak hanya external styles yang dapat digunakan, kamu juga dapat membuat external components, functions, types, dan lainnya yang kemudian dapat diimpor ke file utama agar kode tidak terlalu berantakan dan dapat digunakan kembali (reusable).



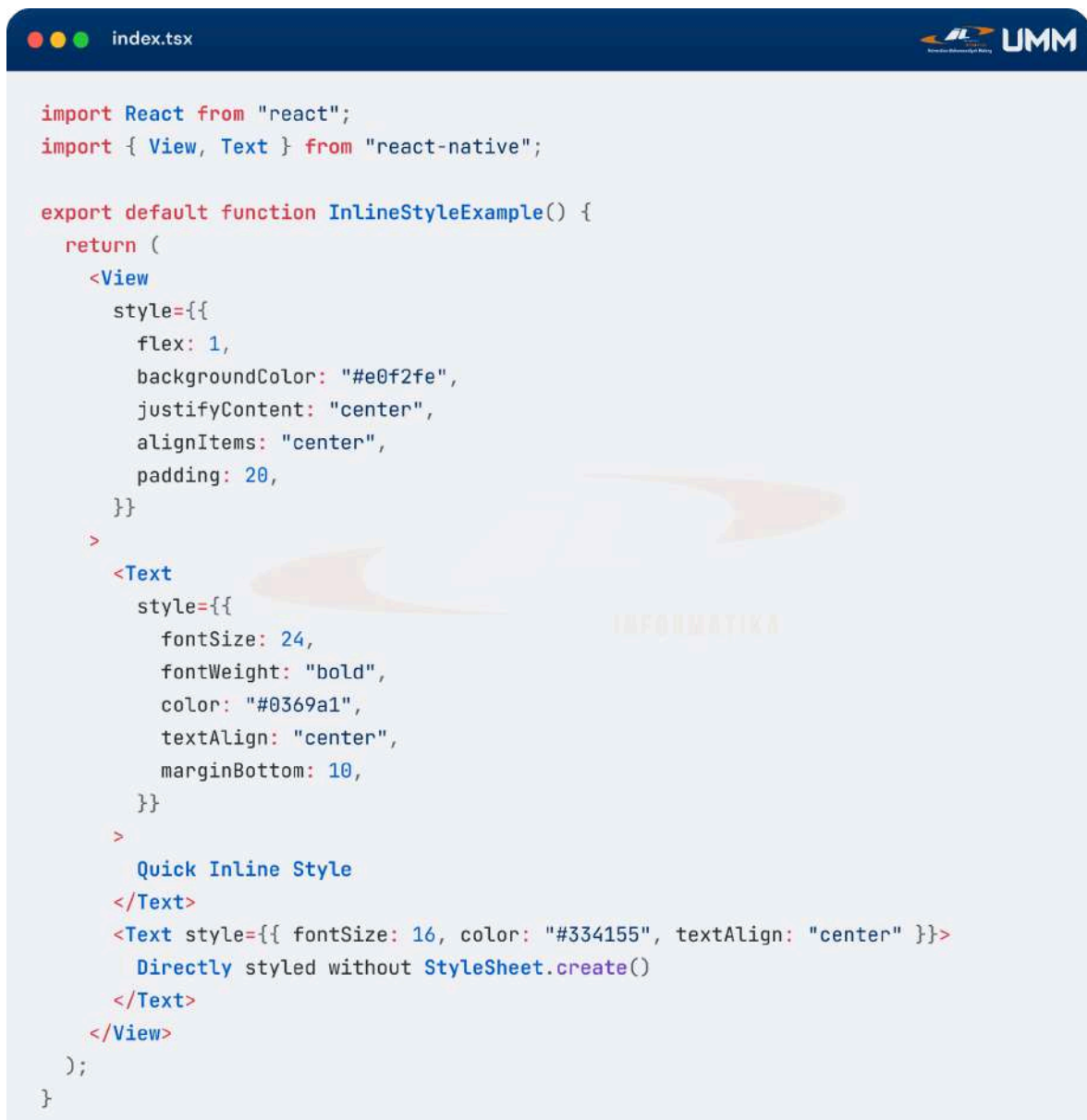
3.3. INLINE STYLING

Inline Styling adalah ketika kamu menuliskan styling secara langsung di dalam tag komponen tanpa membuat objek StyleSheet.

Untuk menulis inline styles, kita menggunakan dua kurung kurawal: `style={{ ... }}`.

- Kurung kurawal bagian luar `{ }` digunakan untuk memberi tahu React bahwa kita sedang masuk ke mode JavaScript.
- Kurung kurawal bagian dalam `{ }` adalah objek JavaScript yang berisi properti-properti styling.

Contoh:



```

import React from "react";
import { View, Text } from "react-native";

export default function InlineStyleExample() {
  return (
    <View
      style={{
        flex: 1,
        backgroundColor: "#e0f2fe",
        justifyContent: "center",
        alignItems: "center",
        padding: 20,
      }}
    >
      <Text
        style={{
          fontSize: 24,
          fontWeight: "bold",
          color: "#0369a1",
          textAlign: "center",
          marginBottom: 10,
        }}
      >
        Quick Inline Style
      </Text>
      <Text style={{ fontSize: 16, color: "#334155", textAlign: "center" }}>
        Directly styled without StyleSheet.create()
      </Text>
    </View>
  );
}
  
```



Kapan sebaiknya menggunakan Inline Styling?

- Sangat cocok untuk pengujian cepat, debugging, atau pembuatan prototipe.
- Berguna ketika styling bergantung langsung pada nilai variabel dinamis (misalnya, `backgroundColor: isSelected ? 'green' : 'gray'`).
- Hindari penggunaan yang berlebihan pada screen yang kompleks karena dapat membuat JSX sulit dibaca dan tidak dioptimalkan sebaik `StyleSheet.create()`.

CODELAB-3 (5-10 menit, kerjakan di lab)

Lanjutkan Codelab 2 sebelumnya dan isi kode di bawah ini. Lengkapi semua bagian yang ditandai dengan “___” dan perbaiki kesalahan jika ada. Pastikan hasil akhirnya dapat berjalan dengan baik tanpa error:

```
example.ts

import {
  View,
  Text,
  TextInput,
  Button,
  _____,
} from "_____";

export default function Index() {
  return (
    <View style={styles.container}>
      <Text style={styles._____}> Hello World </Text>

      <TextInput placeholder="Type here..." style={styles.input} />

      <Button title="Click Me" />
    </View>
  );
}

const styles = _____({
  container: {
    flex: 1,
    backgroundColor: "#e0f2fe",
    justifyContent: "center",
    padding: 20,
  },

  title: {
    _____Size: big,
    fontWeight: "bold",
    color: "red",
    marginBottom: 20,
    textAlign: "center",
  },
});
```

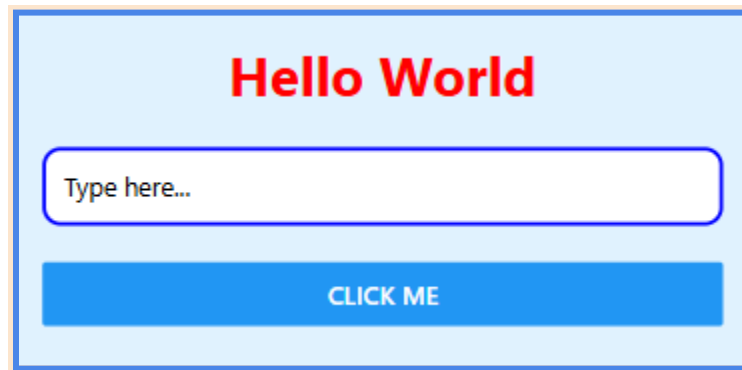


```

-----: {
  borderWidth: 2,
  -----Color: "blue",
  backgroundColor: "white",
  padding: 10,
  borderRadius: 10,
  marginBottom: 20,
}
};

```

Jika semuanya sudah benar dan berhasil dijalankan, hasil tampilannya akan terlihat kurang lebih seperti ini saat aplikasi dijalankan:



Tergantung pada perangkat yang digunakan, mungkin terdapat sedikit perbedaan seperti warna, jarak, atau font, karena beberapa style dapat disesuaikan dengan tema perangkat.

Jika ada materi yang tidak dijelaskan dalam modul ini, silakan gunakan AI, tonton tutorial di YouTube, tanyakan kepada teman, atau cari dokumentasi di Google. Saat ini, informasi dapat ditemukan hampir di mana saja.

Hal yang paling penting adalah kamu mau belajar dan mengeksplorasi sendiri. Jangan takut mencoba hal-hal baru. Belajar pemrograman sering kali melibatkan eksperimen, melakukan kesalahan, dan mencari tahu cara menyelesaikan sesuatu secara bertahap 😊

4. PACKAGE, LIBRARY, & DEPENDENCY

Saat membuat aplikasi React Native, terkadang fitur bawaan saja tidak cukup. Kita mungkin membutuhkan fitur tambahan seperti ikon, navigasi, animasi, grafik, dukungan database, dan masih banyak lagi. Daripada membuat semuanya dari awal, developer biasanya menggunakan library yang dibuat oleh developer lain.

Apa itu Library? Library adalah kumpulan kode siap pakai yang membantu developer menambahkan fitur dengan lebih mudah. Contohnya, daripada membuat ikon secara manual, kita dapat menggunakan library ikon.



Contoh:

```

proejct_mobile2.sh
D:\wira\v1\modul_mobile\proejct_mobile2> npm install @expo/vector-icons
  
```

Library ini menyediakan banyak ikon siap pakai untuk React Native. Kita juga dapat menginstalnya dengan versi yang spesifik seperti: `npm install @expo/vector-icons@15.0.2`. Namun, saya tidak menyarankan cara tersebut. Sebaiknya gunakan tanpa menentukan versi secara spesifik dan biarkan Expo memilih versi yang kompatibel dengan Expo SDK pada project-mu.

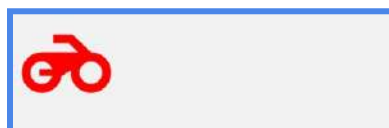
Contoh Penggunaan:

```

example.ts
import { View } from "react-native";
import { MaterialCommunityIcons } from "@expo/vector-icons";

export default function Index() {
  return (
    <View>
      <MaterialCommunityIcons
        name="motorbike"
        size={60}
        color="red"
      />
    </View>
  );
}
  
```

Output:



Apa itu Package? Package adalah modul yang dapat diinstal untuk menambahkan fungsionalitas pada sebuah project. Biasanya, package diinstal menggunakan “npm install” seperti pada contoh sebelumnya. Di sini, “@expo/vector-icons” adalah nama package. Setelah sebuah package diinstal, library tersebut akan ditambahkan ke folder “node_modules” dan siap digunakan.



Apa itu Dependency? Dependency adalah package yang dibutuhkan oleh sebuah project agar dapat berjalan dengan baik. Setelah menginstal sebuah package, package tersebut secara otomatis akan ditambahkan ke file “package.json”.

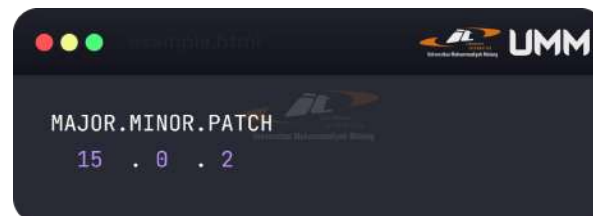
Contoh:



```
...
"dependencies": {
  ...
  "@expo/vector-icons": "^15.0.2"
  ...
}
...
```

Ini berarti project bergantung pada “@expo/vector-icons” versi “^15.0.2” untuk menggunakan ikon. Tanda “^” berarti versi tersebut mengizinkan pembaruan ke versi yang lebih baru yang masih kompatibel, tetapi tidak ke versi mayor yang baru.

Ini memungkinkan versi 15.0.4 | 15.0.5 | 15.1.0 | 15.9.9, tetapi tidak 16.0.0. Karena 16 merupakan versi major baru dan dapat mengandung perubahan yang menyebabkan kode tidak lagi kompatibel. Seperti yang kamu lihat, npm menggunakan Semantic Versioning (SemVer):



Secara umum:

- MAJOR → perubahan yang dapat menyebabkan kode tidak kompatibel (breaking changes)
- MINOR → fitur baru yang tetap kompatibel dengan versi sebelumnya
- PATCH → perbaikan bug

Dan tanpa tanda “^”, berarti kita harus menggunakan versi 15.0.2 yang tepat. Apa yang terjadi jika kita memaksakan versi tersebut? Apakah program hanya tidak dapat berjalan atau akan muncul error lainnya? Tidak selalu. Program mungkin tetap berjalan, tetapi dapat menyebabkan masalah kompatibilitas, error yang tidak terduga, atau bahkan membuat aplikasi tidak dapat berjalan sama sekali.



Namun, apa pun yang terjadi, tanpa dependency tersebut sama sekali, fitur ikon tidak akan berfungsi.

Analogi sederhana: bayangkan kita sedang membangun rumah.

- Project = Rumah
- Library = Toko furnitur
- Package = Barang furnitur
- Dependency = Furnitur yang dibutuhkan oleh rumah

Ketika bekerja sendiri, menginstal sebuah package itu sederhana. Kamu hanya perlu menjalankan perintah `"npm install @expo/vector-icons"`. Namun, dalam project nyata, developer biasanya bekerja dalam tim menggunakan **Git**.

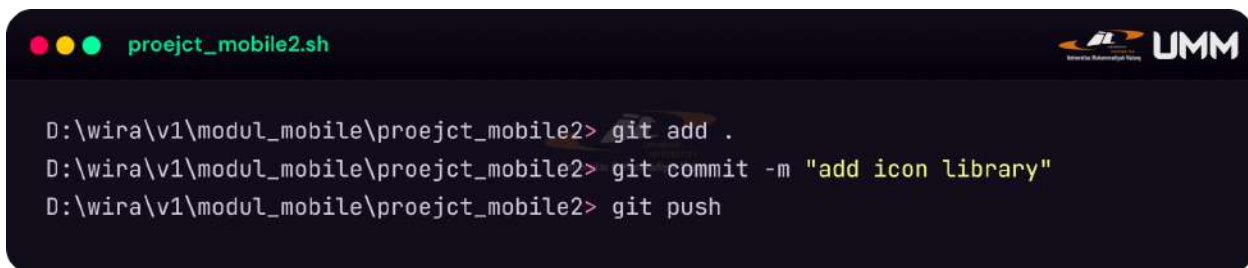
Mari kita bayangkan kamu memiliki sebuah tim dengan peran seperti ini:



Image 4.1

Bayangkan kamu menginstal package berikut: `npm install @expo/vector-icons`. Setelah instalasi, informasi package tersebut akan secara otomatis disimpan dalam "package.json" dan folder "node_modules". Ini berarti project membutuhkan "@expo/vector-icons" agar dapat berjalan dengan baik.

Apa yang terjadi di GitHub? Setelah menginstal package, kamu melakukan push project ke GitHub.



```

D:\wira\v1\modul_mobile\proejct_mobile2> git add .
D:\wira\v1\modul_mobile\proejct_mobile2> git commit -m "add icon library"
D:\wira\v1\modul_mobile\proejct_mobile2> git push
  
```

Kemudian, developer lain melakukan pull request untuk project terbaru.



```

proejct_mobile2.sh
D:\wira\v1\modul_mobile\proejct_mobile2> git pull

```

Mereka akan mendapatkan **source code**, **package.json**, dan **informasi dependency**. Namun, biasanya mereka **tidak mendapatkan folder “node_modules”**.

Mengapa? Karena folder **“node_modules”** berukuran sangat besar dan biasanya diabaikan dengan **.gitignore**.

Jadi, Bagaimana Developer Lain Mendapatkan Package-nya?

Setelah melakukan *clone* atau *pull* project, mereka hanya perlu menjalankan **“npm install”** atau **“npm i”** di terminal.

Perintah ini akan secara otomatis menginstal semua dependency yang diperlukan dan tercantum dalam **package.json**, termasuk **@expo/vector-icons**. Jadi, mereka tidak perlu menginstal package satu per satu.

Ada banyak library yang tersedia dan dibuat oleh developer lain. Tidak mungkin untuk membahas semuanya dalam modul ini, tetapi beberapa library yang paling sering digunakan dalam React Native adalah:

- **@expo/vector-icons** → untuk ikon
- **react-navigation / expo-router** → untuk navigasi antar-screen
- **axios** → untuk melakukan request ke API
- **react-native-paper** → untuk komponen UI
- **react-native-reanimated** → untuk animasi

Sebelum menggunakan *library* apa pun, pastikan **membaca dokumentasinya terlebih dahulu**. Setiap library dapat memiliki langkah instalasi, persyaratan konfigurasi, dan cara penggunaan yang **berbeda**.



CODELAB-4 (5-10 menit, kerjakan di lab)

Tugasmu adalah melanjutkan dari codelab sebelumnya dan membuat hasil tampilannya terlihat seperti ini:



Langkah-langkah: Import Ionicons dari @expo/vector-icons, kemudian gunakan ikon dengan nama "information-circle" dan "hand-left".

Kamu dapat menggunakan *stylesheet* dari tautan yang disediakan [di sini](#).

5. SINTAKS DASAR TYPESCRIPT

Sama seperti bahasa pemrograman lainnya, React Native juga memiliki konsep-konsep dasar pemrograman seperti variabel, fungsi, perulangan, kondisi, dan masih banyak lagi.

Untuk saat ini, dalam modul ini kita akan berfokus untuk mempelajari dasar-dasar pemrograman terlebih dahulu secara bertahap, sehingga kamu dapat memahami cara kerja kode React Native sebelum beralih ke konsep yang lebih lanjut.

5.1. VARIABLE

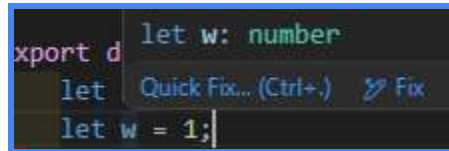
Variable (variabel) adalah wadah yang digunakan untuk menyimpan data. Variabel membantu kita menyimpan nilai yang dapat digunakan kembali dalam kode. Dalam JavaScript/TypeScript, variabel umumnya dibuat menggunakan `const` dan `let`:

- **Const** digunakan untuk nilai yang **tidak boleh di-reassign** (diberi nilai baru).
- **Let** digunakan untuk nilai yang **dapat berubah**. Kamu juga nantinya dapat menggunakan operasi matematika seperti `"*"`, `"+"`, `"-"`, `"/"`, dan sebagainya.



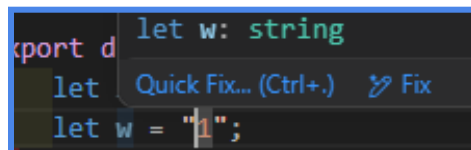
Saat menggunakan `const` atau `let`, kita tidak perlu menentukan tipe data secara eksplisit. TypeScript dapat menentukan tipe data secara otomatis berdasarkan nilai yang diberikan pada variabel.

Sebagai contoh, jika kamu menulis `let w = 1;`, TypeScript akan menganggapnya sebagai **number**:



```
export default function() {
  let w: number
  let w = 1;
```

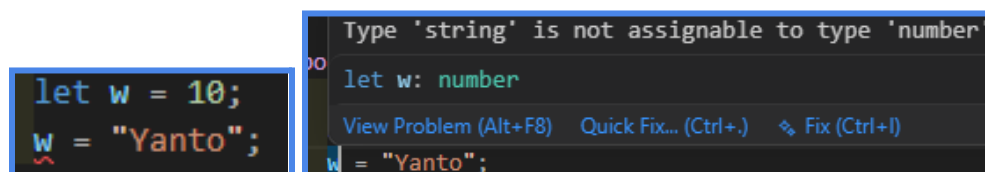
Sedangkan `let w = "1";` akan dianggap sebagai **string**.



```
export default function() {
  let w: string
  let w = "1";
```

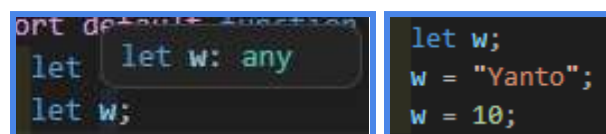
TIPS: Kamu dapat mengarahkan cursor mouse ke sebuah variabel untuk melihat **tipe data** atau **informasi** lain tentang variabel tersebut. Di **Visual Studio Code** atau IDE serupa, fitur ini biasanya tersedia dan sangat membantu.

Kemudian, setelah kita menulis `let w = 1;`, TypeScript secara otomatis menentukan tipenya sebagai **number**, dan tipe tersebut **tidak dapat diubah setelahnya**. Jadi, variabel yang dideklarasikan dengan `let` dapat diberi **nilai baru**, tetapi **tipe datanya tidak dapat diubah**.



```
let w = 10;
w = "Yanto";
```

Namun, jika kita tidak memberikan nilai awal, tipe datanya dapat berubah secara otomatis menjadi **"any"**. Sebagai contoh:



```
let w;
w = "Yanto";
w = 10;
```

any pada dasarnya berarti TypeScript tidak menerapkan batasan tipe apa pun terhadap nilai tersebut, sehingga nilainya dapat berupa apa saja saat *runtime*. Namun, tipe **any** **tidak disarankan** karena menonaktifkan pemeriksaan tipe (*type checking*) TypeScript, sehingga banyak error baru akan terdeteksi saat program dijalankan.



Kamu dapat menggunakan kata kunci **"typeof"** untuk melihat tipe data sebuah variabel saat *runtime*:

```
example.ts

const x = "Hello";
console.log(x); // output: Hello
console.log(typeof x); // output: string

function someFunction() { return "Hello" }
console.log(someFunction); // output: [Function: someFunction]
console.log(someFunction()); // output: Hello
console.log(typeof someFunction); // output: function
console.log(typeof someFunction()); // output: string
```

Namun, dalam *TypeScript*, kita dapat menentukan tipe data sebuah variabel daripada membiarkan *TypeScript* menentukannya secara otomatis. Sebagai contoh:

```
let name: string = "Joko".
```

Cara ini lebih eksplisit dibandingkan dengan hanya menulis:

```
let name = "Joko"
```

Kita juga dapat menggunakan **union type** ketika sebuah variabel dapat memiliki lebih dari satu tipe yang diperbolehkan. Sebagai contoh:

```
let name: string = null.
```

Tidak diperbolehkan. Namun:

```
let name: string | null = null.
```

Diperbolehkan.

Hal ini berguna ketika sebuah variabel dapat berisi **string** atau **kosong (null)**.

```
example.ts

1      2
let variableName : variableType; ✓

1      2
let name : string; ✓

1      2
let age : number = 67; ✓
```



```

      1      2
let productCode : number | string; ✓

      1      2      1      2
let varA : number, varB : string; ✓

```

Contoh Penggunaan:

```

example.ts

import { View, Text } from "react-native";

export default function Index() {
  const appName = "My First App";
  let total = 10;
  total = total + 5;

  return (
    <View>
      <Text> {appName} </Text>
      <Text> Total: {total} </Text>
    </View>
  );
}

```

Output:

```

My First App
Total: 15

```

Biasakan untuk menulisnya dengan cara tersebut, karena beberapa fungsi dapat melakukan perbandingan ketat (strict comparison) menggunakan "===". Sebagai contoh, 60 dan "60" dapat dianggap sama meskipun memiliki tipe data yang berbeda. Perhatikan kode berikut:




```
example.ts

let varA: string = "60";
let varB: any = 60;

console.log(varA == varB); // true
console.log(varA === varB); // false

console.log(typeof varA); // string
console.log(typeof varB); // number
```

Selain itu, JavaScript dan TypeScript bersifat Case Sensitive. Artinya, huruf uppercase dan lowercase dianggap sebagai nama yang berbeda.

5.2. CONDITIONS

Sebuah kondisi (*condition*) pada dasarnya adalah cara untuk membuat keputusan dalam kode.

Contoh:

```
example.ts

import { View, Text } from "react-native";

export default function Index() {

  const score = 85;
  let grade = "";
  const isLogin = true;

  // if statement (basic)
  if (score >= 90) { grade = "A"; }
  else if (score >= 75) { grade = "B"; }
  else { grade = "C"; }

  return (
    <View>
      { /* if statement (basic) */ }
      <Text> Grade: {grade} </Text>

      { /* Ternary Operator (? :) */ }
      <Text> Greetings: {isLogin ? "Welcome" : "Please Login"} </Text>
    </View>
  );
}
```



Output:

Grade: B
Greetings: Welcome

Artinya: jika nilainya **lebih besar dari atau sama dengan 90**, maka **grade = "A"**. Namun, jika nilainya **lebih besar dari atau sama dengan 75**, maka **grade = "B"**. Jika **tidak ada satu pun kondisi yang terpenuhi**, maka **grade = "C"**. Kamu sudah mempelajari hal ini sebelumnya.

Untuk *ternary operation*, artinya jika **isLogin bernilai true**, maka akan menghasilkan **"Welcome"** (bagian kiri). Sebaliknya, jika bernilai **false**, maka akan menampilkan **"Please Login"** (di bagian kanan).

Atau kamu juga dapat menggunakan **switch case**:

```
example.ts

const day: number = 2;
switch (day) {
  case 1:
    console.log("Monday"); break;
  case 2:
    console.log("Tuesday"); break;
  case 3:
    console.log("Wednesday"); break;
  default:
    console.log("Invalid day");
}
```

5.3. FUNCTIONS

Sebuah **function (fungsi)** adalah **blok kode yang menjalankan tugas tertentu**. Fungsi membantu kita menghindari penulisan kode yang sama berulang kali. **Struktur dasar:**

```
example.ts

// Basic function (using 'function' keyword):
function sayHello() { ... }

// Arrow function (using 'const | let' keyword):
const sayHello = () => { ... }
```

Pada *arrow function*, kamu juga dapat **memberikan parameter**, seperti:

const sayHello = (param1, param2) => {}.



Atau, jika hanya memiliki satu parameter:

`const sayHello = param => {}.`

Kamu juga dapat menggunakan **inline function** dengan **arrow function**. *Inline function* adalah fungsi yang ditulis langsung di tempat fungsi tersebut digunakan. Contoh:

```
example.ts

// Instead of declaring a function
const sayHello = () => { Alert.alert("Hello"); };

// And then passing it later
onPress={sayHello}

// We can rather create and pass that function at the same time
onPress={() => { Alert.alert("Hello"); }}
```

Hal ini berguna ketika kamu hanya ingin membuat fungsi sederhana yang digunakan sekali. Jadi, kamu tidak perlu mendeklarasikan banyak fungsi. Hal ini membuat kode lebih sederhana.

Perlu diperhatikan bahwa menulis arrow function mirip dengan memberikan/meneruskan fungsi (passing the function), bukan memanggil fungsi (calling the function). Apa perbedaannya?

Dalam JavaScript, terdapat perbedaan penting antara memanggil fungsi seperti `functionX()` dengan tanda kurung, dan `functionX` tanpa tanda kurung:

- **functionX():** Artinya, memanggil/menjalankan fungsi sekarang juga.

```
example.ts

function sayHello() { console.log("Hello"); }

const myFunction = sayHello(); // output: Hello
```

- Kode:
- Kamu mengatakan: **"Jalankan sayHello."**
- Output: **Hello**
- **functionX:** Tanpa (), kamu sedang merujuk pada fungsi itu sendiri, bukan menjalankan fungsi tersebut.



```
example.ts

function sayHello() { console.log("Hello"); }

const myFunction = sayHello; // no output
```

- o Kode:
- o Kamu **belum memanggil fungsi tersebut**, tetapi hanya menetakannya atau meneruskannya.
- o **Output: *kosong**
- o Kamu dapat menjalankannya nanti menggunakan: `myFunction();`

Ada dua jenis fungsi: **(1) Already Defined Functions:** fungsi bawaan yang telah didefinisikan dan disediakan oleh library atau framework. **(2) Custom Functions:** fungsi yang kita buat sendiri untuk menjalankan tugas tertentu.

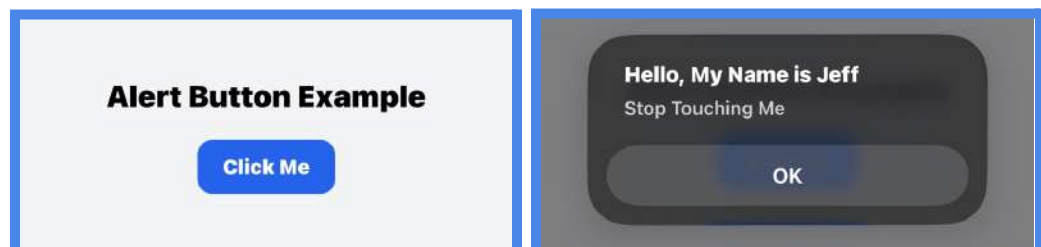
A. FUNCTION BAWAAN:

```
example.ts

export default function Index() {
  const showAlert = () => {
    Alert.alert("Hello, My Name is Jeff", "Stop Touching Me");
  };
  return (
    <View style={styles.container}>
      <Text style={styles.title}> Alert Button Example </Text>

      <Pressable style={styles.button} onPress={showAlert} >
        <Text style={styles.buttonText}> Click Me </Text>
      </Pressable>
    </View>
  );
}
```

Output:



Alert function adalah **fungsi bawaan (built-in function)**. Namun, di sini kita menempatkannya **dalam** fungsi yang **kita buat sendiri**, sehingga di dalam komponen kita cukup meneruskannya secara langsung.

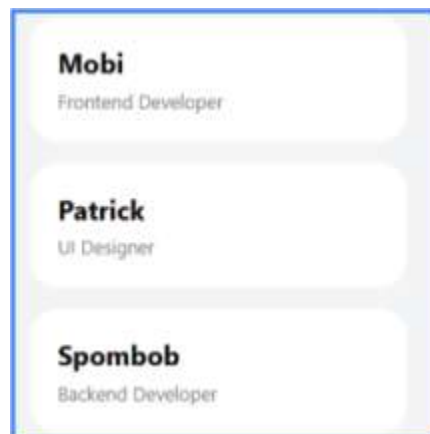
Jadi, hanya ketika tombol ditekan, fungsi custom kita akan dipanggil, dan di dalam fungsi tersebut, *Alert function* akan dijalankan.

B. FUNCTION CUSTOM:

```
example.tsx
export default function Index() {
  // Custom Card Function
  const renderProfileCard = (name, role) => {
    return (
      <View style={styles.card}>
        <Text style={styles.name}>{name}</Text>
        <Text style={styles.role}>{role}</Text>
      </View>
    );
  };

  return (
    <View style={styles.container}>
      {renderProfileCard("Mobi", "Frontend Developer")}
      {renderProfileCard("Patrick", "UI Designer")}
      {renderProfileCard("Sponbob", "Backend Developer")}
    </View>
  );
}
```

Output:



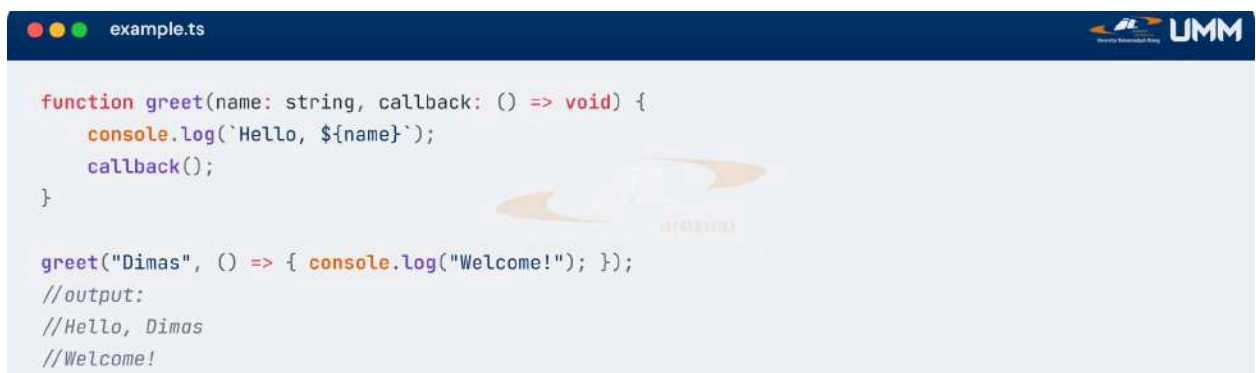
Contoh kedua adalah *custom function* yang kita buat sendiri.

Daripada **membuat komponen satu per satu secara manual**, sering kali lebih baik membuat sebuah fungsi yang hanya perlu kita isi parameternya dengan argument, kemudian komponen akan dibuat secara otomatis berdasarkan definisi yang telah kita buat.

Pendekatan ini sangat berguna untuk membuat sesuatu seperti tampilan *card*, terutama jika dikombinasikan dengan *loop*, sehingga jumlah *card* dapat dibuat secara dinamis dan tidak perlu ditulis secara *hardcoded*.

C. CALLBACK FUNCTION (BONUS)

Callback adalah sebuah fungsi yang **diberikan sebagai argumen kepada fungsi lain agar dapat dipanggil nanti**. Fungsi yang menerima *callback* menentukan **kapan** dan **bagaimana** fungsi tersebut dijalankan. *Callback* umum digunakan dalam pemrograman *asynchronous* ketika kita ingin menjalankan kode tertentu setelah suatu operasi selesai.



```
example.ts

function greet(name: string, callback: () => void) {
  console.log(`Hello, ${name}`);
  callback();
}

greet("Dimas", () => { console.log("Welcome!"); });
//output:
//Hello, Dimas
//Welcome!
```

TIPS: Gunakan *template literal* ``. Cara kerjanya seperti *string* biasa, tetapi kamu dapat menyisipkan variabel di antara tanda *backtick* dengan *\${}*.

TIPS: Gunakan *() => <type>* untuk mendeskripsikan fungsi yang mengembalikan tipe tertentu.

5.4. LOOPS

Dalam React Native, terdapat beberapa cara untuk mengulang data (*looping*). Namun, beberapa metode lebih sering digunakan daripada yang lainnya. Berikut adalah metode *looping* yang paling umum:

A. MAP()

map() digunakan untuk melakukan iterasi pada setiap data dalam sebuah array dan menghasilkan sesuatu dari setiap data tersebut. Fungsi ini juga merupakan salah satu



array method dalam JavaScript, bersama dengan filter(), find(), sort(), reduce(), dan masih banyak lainnya.

Sebagai contoh, misalkan kita memiliki:

```
location.tsx
const fruits = ["Apple", "Banana", "Orange"];
```

Dan kita ingin menampilkan semua buah tersebut di layar.

Daripada menuliskannya satu per satu:

```
location.tsx
<Text>{fruits[0]}</Text>
<Text>{fruits[1]}</Text>
<Text>{fruits[2]}</Text>
```

Kita dapat menggunakan map().

Pola sintaksnya adalah:

`someArray.map(someFunction)`

Misalkan someFunction berisi:

`(passTheArrayHere) => (<Text> {passTheArrayHere} </Text>)`

Maka, ketika digabungkan, hasilnya dapat terlihat seperti ini:

`someArray.map((passTheArrayHere) => (<Text>{passTheArrayHere}</Text>))`

Berikut adalah contoh cara menggunakannya:

```
index.html
<View style={styles.container}>
  {fruits.map(
    (fruit) => ( <Text>{fruit}</Text> )
  )}
</View>
```

TIPS: Kita menggunakan `() => ()` daripada `() => {}`, yang berarti `()` memungkinkan implicit return, sedangkan `{}` memerlukan pernyataan return secara eksplisit.



Output:

```
Apple
Banana
Orange
```

Jika diperhatikan dengan saksama, terdapat **"fruit"** dan **"fruits"**. Apa perbedaannya?

fruits adalah array yang ingin kita lakukan *looping*, sedangkan **fruit** adalah nilai dari setiap item dalam array pada setiap iterasi.

Jadi, pada iterasi pertama, **fruit = "Apple"**.

Pada iterasi kedua, **fruit = "Banana"**.

Pada iterasi ketiga, **fruit = "Orange"**.

Pada dasarnya, ini sama seperti mengakses **fruits[0]**, **fruits[1]**, dan **fruits[2]**.

Jadi, misalnya, jika kamu mengubah *fruit* yang berisi **"Apple"** menjadi **"Ringo"**, array *fruits* asli tidak akan berubah. Hanya variabel **fruit** yang berubah.

Kita juga dapat menggunakan parameter kedua, yaitu **index**:

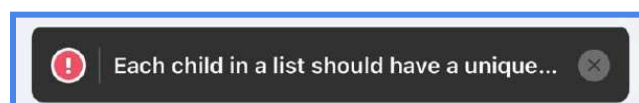
```
index.html

<View style={styles.container}>
  {fruits.map(
    (fruit, index) => ( <Text> {index} : {fruit}</Text> )
  )}
</View>
```

Output:

```
0 : Apple
1 : Banana
2 : Orange
```

Jika kamu mencoba kode di atas, kemungkinan besar akan mendapatkan peringatan berikut:



Each child in a list should have a unique "key" prop. Artinya, setiap *child* dalam sebuah list harus memiliki *prop* "**key**" yang unik.

Jadi, kita harus memberikan *key*:

```
location.tsx

const fruits = [
  {"id": 1, "name": "Apple"},
  {"id": 2, "name": "Banana"},
  {"id": 3, "name": "Orange"}
];
```

Di sini, kita akan menggunakan "**id**" sebagai *key*.

Apa itu *key*? *Key* adalah **identifikasi unik yang digunakan oleh React untuk mengidentifikasi setiap item dalam sebuah list.**

Kita dapat menggunakan *key* pada sebuah komponen seperti ini:

```
location.tsx

<View style={styles.container}>
  {fruits.map(
    (fruit, index) => ( <Text key={fruit.id}> {index} : {fruit.name} </Text> )
  )}
</View>
```

Mengapa kita membutuhkan *key* daripada hanya menggunakan *index*? Alasannya adalah karena *index* dapat berubah ketika data ditambahkan atau dihapus. Dengan menggunakan *key*, React tetap dapat mengidentifikasi item mana yang mana, meskipun posisi item tersebut dalam array berubah. Dengan begitu, peringatan tersebut seharusnya sudah hilang.

B. <FLATLIST> (BUKAN LOOP TAPI GUNANYA SAMA)

Meskipun `.map()` sangat bagus untuk list berukuran pendek, menampilkan ratusan item menggunakan `.map()` dapat membuat aplikasi menjadi lambat karena semuanya akan dirender sekaligus ke dalam memori.

FlatList adalah komponen khusus yang berkinerja tinggi dan sudah tersedia di React Native. Komponen ini hanya *me-render* item yang sedang terlihat di layar (*lazy loading*).



Jika kamu sudah memahami `map()`, mempelajari **FlatList** sebenarnya cukup mudah karena konsepnya serupa: kita memiliki sebuah array → kemudian React Native membuat sebuah komponen untuk setiap data.

FlatList membutuhkan 3 props utama:

1. **data**: Array data yang ingin kamu tampilkan.
2. **renderItem**: Fungsi yang mengembalikan komponen untuk setiap item.
3. **keyExtractor**: Fungsi yang mengambil *key* unik untuk setiap item.

Pertama, import komponen tersebut seperti ketika kamu meng-*import* komponen lain seperti `<View>`. Kemudian, kamu dapat menggunakannya seperti ini:



```
location.tsx
<FlatList
  data={fruits}
  keyExtractor={ (item) => item.id }
  renderItem={
    ({ item }) => ( <Text>{item.name}</Text> )
  }
/>
```

Item di sini diambil dari nilai yang kamu berikan pada data. Dalam kasus ini, `{fruits}`.

C. PRIMITIVE LOOP

Apakah kita dapat menggunakan loop tradisional seperti `for`, `for...of`, atau `while`? Ya, tetapi tidak secara langsung di dalam blok `return (...)` JSX, karena JSX hanya menerima JavaScript expressions, bukan statements!

Ini salah:



```
location.tsx
return (
  <View>
    for (let i = 0; i < fruits.length; i++) {
      <Text>{fruits[i]}</Text>
    }
  </View>
);
```

Untuk menggunakan loop primitif, kamu perlu menjalankan loop tersebut di luar/sebelum pernyataan `return`, memasukkan komponen yang dihasilkan ke dalam array, lalu me-render array tersebut di dalam JSX.



Contoh:

```
location.tsx

import { View, Text } from "react-native";

export default function App() {
  const fruits = ["Apple", "Banana", "Orange"];
  const fruitComponents = [];

  for (let i = 0; i < fruits.length; i++) {
    fruitComponents.push(
      <Text key={i}>{fruits[i]}</Text>
    );
  }

  return (
    <View> {fruitComponents} </View>
  );
}
```

Atau:

```
location.tsx

import { View, Text } from "react-native";

export default function App() {
  const fruits = ["Apple", "Banana", "Orange"];
  const fruitComponents = [];
  let i = 0;

  while (i < fruits.length) {
    fruitComponents.push(
      <Text key={i}>{fruits[i]}</Text>
    ); i++;
  }

  return (
    <View> {fruitComponents} </View>
  );
}
```



Output:

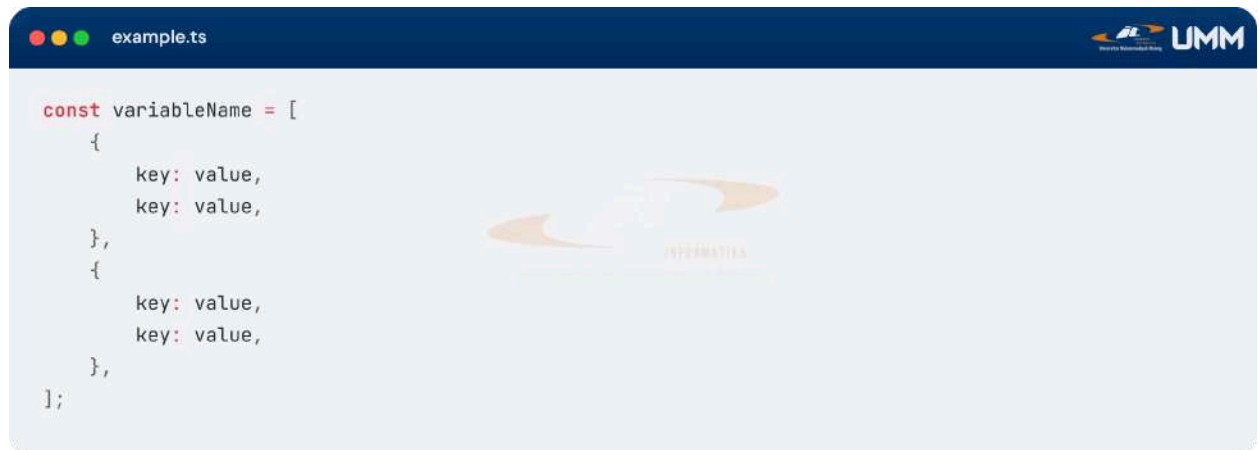
Apple
Banana
Orange

5.5. VARIABLE LANJUTAN

A. ARRAY OF OBJECT

Terkadang kita perlu menyimpan **banyak data yang saling berkaitan**. Contohnya: *banyak makanan*, *banyak mahasiswa*, atau *banyak produk*. Untuk hal ini, kita biasanya menggunakan **Array + Object**, atau yang umum disebut **Array of Objects**.

Format penulisannya terlihat seperti ini:



```
example.ts
const variableName = [
  {
    key: value,
    key: value,
  },
  {
    key: value,
    key: value,
  },
];
```

[...] artinya: array (sebuah daftar data).

{ ... } artinya: object (satu item data).




```
example.ts

const foods = [
  {
    id: "1",
    name: "Burger",
    price: 25000,
  },
  {
    id: "2",
    name: "Pizza",
    price: 30000,
  },
];
```

Kamu juga dapat mengisi array objek yang kosong menggunakan *loop* seperti ini:

```
example.tsx

const foods = [];

for (let i = 1; i <= 5; i++) {
  foods.push({
    id: `${i}`, // 1, 2, 3, ...
    name: `Food ${i}`, // Food 0, Food 1, Food 2, ...
    price: i * 10000, // 10000, 20000, 30000, ...
  });
}
```

TIPS: Gunakan `push()` untuk menambahkan elemen baru ke bagian akhir array.

Jika kamu ingin mengakses nilainya, kamu dapat menggunakan contoh berikut:

```
example.ts

console.log(foods[0]); //output: { id: '1', name: 'Burger', price: 25000 }

console.log(foods[0].name); //output: Burger
console.log(typeof foods[0].name); //output: string
```

B. TYPE & INTERFACE

Apa itu **type**? *Type* adalah **tipe data** yang dapat kita berikan pada variabel.

Apa itu **interface**? *Interface* dalam TypeScript adalah cara untuk mendefinisikan **bentuk** atau **struktur suatu objek**. Interface memberi tahu TypeScript properti apa saja yang



harus dimiliki sebuah objek dan tipe data apa yang harus dimiliki oleh setiap properti tersebut. (*type* juga dapat berfungsi seperti *interface*, dengan definisi yang serupa).

Contoh:

```
example.ts

type foodsType = "food" | "drink" | "snack" | "dessert" ;

interface FoodsObject{
  readonly id: string | number, // readonly means that the property cannot be changed after it is set
  name: string,
  price: number,
  type?: foodsType ; // Sign "?" means that the property is optional.
}

const foods: FoodsObject[] = [
  {
    id: "1",
    name: "Burger",
    price: 25000,
  },
  {
    id: 2,
    name: "Pizza",
    price: 30000,
    type: "food",
  },
] as const; // "as const" means that the array is read-only and cannot be modified.
```

Tipe **foodsType** adalah *union type*, yang berarti properti *type* hanya dapat berisi salah satu dari empat nilai tertentu: "food", "drink", "snack", atau "dessert".

Interface **FoodsObject** mendefinisikan struktur yang harus diikuti oleh setiap objek di dalam array *foods*.

Properti **id** dapat berupa *string* atau *number*.

Sementara itu, *name* dan *price* adalah properti yang **wajib diisi**, sedangkan properti **type** bersifat *opsional* karena menggunakan "?", sehingga sebuah objek dapat memiliki properti tersebut atau tidak.

Kata kunci **readonly** pada *id* berarti nilainya tidak dapat diubah setelah objek dibuat.

Variabel **foods** adalah sebuah **array of FoodsObject**, yang berarti setiap elemennya **harus mengikuti** struktur yang telah didefinisikan oleh *interface*.



As const di bagian akhir memberi tahu *TypeScript* untuk memperlakukan seluruh array beserta isinya sebagai *readonly* dan mempertahankan nilai literalnya. Namun, dalam kode ini, as const sebenarnya **tidak diperlukan** dan bertentangan dengan anotasi `FoodsObject[]` karena **as const** membuat array menjadi *read-only*. Jika kamu hanya ingin properti `id` tidak dapat diubah, penggunaan *readonly* pada *interface* sudah cukup.

Hal ini membuat kode **lebih mudah** dipahami oleh programmer ketika ingin mengetahui arti dari suatu nilai. Percayalah, ini akan berguna nantinya.

Hal ini juga membantu **IDE**, seperti *Visual Studio Code*, dalam memberikan fitur **autocomplete**.

CODELAB-5 (5-10 menit, kerjakan di lab)

Ingin mencoba dan bereksperimen dengan *TypeScript* sendiri, tetapi menjalankan program Expo membutuhkan waktu terlalu lama? Sebenarnya, kamu tidak memerlukan Expo atau React untuk berlatih *TypeScript*. Berikut caranya:

Pertama, buka folder project-mu, atau folder apa pun yang kamu inginkan. Kemudian, buat sebuah file di dalam folder tersebut dengan nama `exercise.ts`.

Tulis kode *TypeScript*-mu di dalam file tersebut. Ketika ingin menjalankannya, buka terminal dan jalankan: `node exercise.ts`

Setelah selesai, lengkapi kode di bawah ini. Pastikan tidak ada error.

```

interface Student {
  ____: string;
  age: ____;
  score: number;
}

const students: ____[] = [
  {
    name: Andi,
    age: 20,
    score: 85
  },
  {
    name: "Budi",
    age: 19,
    score: 70
  },
]
  
```



```

{
  name: "Citra",
  age: "21",
  ____: 90
}
];

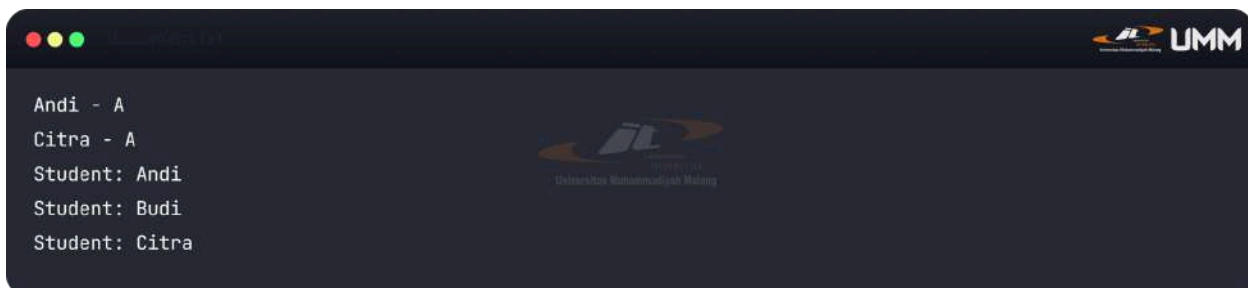
function getGrade(score: number): string {
  if (score >= 80) { return "A"; }
  ____ (score >= 70) { return "B"; }
  else { return "C"; }
}

students.map(student => {
  const grade = getGrade(student.score);
  if (student.age >= "20") {
    console.log( ____ .name + " - " + grade );
  }
});

let i: number = 0;
while (i < students.length) {
  console.log("Student:", students[i].name);
  i++;
}

```

Output yang diharapkan:



```

Andi - A
Citra - A
Student: Andi
Student: Budi
Student: Citra

```



TUGAS PEKAN DEMO

1. DESKRIPSI

1. Buat kelompok & pilih tema:

- Buat kelompok yang terdiri dari 3 anggota dan pilih tema project kalian sendiri melalui tautan [di sini](#). *(Tema project harus unik di seluruh kelas.) Jika terdapat dua kelompok yang memilih tema yang sama, kelompok yang mendaftarkannya lebih dulu akan mendapatkan tema tersebut.*
- Untuk tugas pada modul-modul berikutnya, kalian akan bekerja bersama anggota kelompok di rumah untuk mengembangkan project berdasarkan tema tersebut.
- Untuk melihat kontribusi setiap anggota kelompok, pastikan kalian bekerja bersama menggunakan lingkungan GitHub agar kami dapat melihat bahwa beban kerja dan tugas telah dibagi secara adil kepada semua anggota (tidak ada satu orang yang mengerjakan semuanya sendirian).
- Tugas ini akan berlanjut sepanjang Modul 1–6. Kalian akan terus mengembangkan project yang sama dari modul sebelumnya. Pastikan semua modul sebelumnya dapat berjalan dengan baik karena semuanya juga akan diperiksa pada akhir perkuliahan.

2. Buat aplikasi kalian bersama:

- Tugas kalian adalah menerapkan semua yang telah dipelajari dalam modul ini ke dalam satu aplikasi yang dapat berjalan (**perhatikan kriteria penilaian dengan saksama**).
- Pastikan aplikasi kalian mengikuti tema yang telah dipilih.
- Kalian bebas menerapkan materi yang telah dipelajari di bagian mana pun dalam kode. Pastikan penerapannya masuk akal dan dilakukan dengan benar.
- Kalian tidak perlu melakukan *build* aplikasi. Namun, jika ingin melakukannya, tentu saja diperbolehkan. Kalian juga dapat menggunakan Expo Go dengan menjalankan: `npx expo start --go`.
- Saya menyediakan **contoh** aplikasi dasar berikut: [Tautan](#).

3. Di pekan demo, kalian perlu:

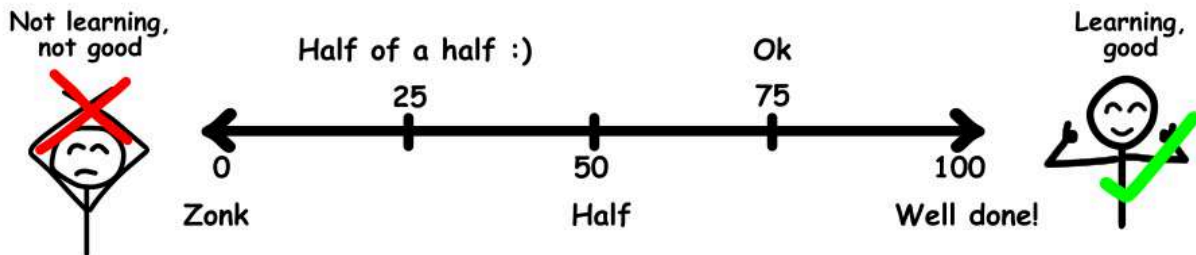
- Menunjukkan hasil pekerjaan kalian kepada asisten praktikum (**secara individu**).
- Menjawab pertanyaan yang diberikan oleh asisten praktikum (**secara individu**).
- Melakukan modifikasi kode secara langsung sesuai permintaan asisten praktikum (**secara individu**).
- Praktikan tidak diperkenankan membuka modul atau menggunakan fitur AI pada code editor (inline maupun chatbot agent).



2. KRITERIA PENILAIAN

Assessment Criteria	Assessment Weight
Codelab 1-5	40%
Tugas Pekan Demo	60%
Menerapkan Deklarasi Custom Function & Loop.	10%
Menerapkan Type & Array of Objects.	10%
Menerapkan Inline & External Styles.	10%
Menjawab Pertanyaan.	15%
Menyelesaikan Modifikasi Kode.	15%
TOTAL	100%

3. RENTANG PENILAIAN



KESIMPULAN AKHIR MODUL

Kenapa kalian tetap wajib mengerjakan codelab secara live di lab? Karena salah satu tujuan praktikum adalah agar kalian bisa praktik langsung dan saling membandingkan dan berdiskusi mengenai cara kalian mengerjakan sesuatu dengan teman di sebelah kalian.

Bisa saja cara yang kalian gunakan sama, tetapi hasilnya berbeda. Atau sebaliknya, cara kalian berbeda, tetapi ternyata menghasilkan output yang sama. Nah, dari situlah sesi diskusi antarsesama praktikan bisa muncul.

"Eh, punya kamu kok begini? Punyaku kok begitu?"

"Oh, ternyata isi configuration.json-nya berbeda."

"Oh, ternyata extension yang digunakan berbeda."

"Eh, aku menemukan cara yang lebih efisien di YouTube."

Diskusi-diskusi seperti itulah salah satu alasan kenapa kalian tetap perlu mengerjakan codelab di lab, meskipun sebelumnya kalian sudah mengerjakannya di rumah.

Kemudian, misalnya saat mengerjakan di lab, hasil pekerjaan kalian ternyata gagal atau mengalami kendala, itu tidak masalah. Kalian tetap boleh menunjukkan kepada asisten versi yang sudah kalian kerjakan di rumah. Itu diperbolehkan.

Karena pada akhirnya ini adalah praktikum, bukan kelas teori. Jadi, selain menyelesaikan codelab, manfaatkan juga waktu di lab untuk berdiskusi, membandingkan cara kerja, dan belajar dari praktikan lain.

